

Kubernetes vs Docker Swarm: Choosing the Right Orchestration Tool



This overview of the differences between Kubernetes and Docker Swarm will help DevOps developers determine the right container orchestration tool for their project.



Let's assume your app is a huge success with many people using it, and you need to run it on multiple servers to handle the large usage. This means there will be a lot of containers to manage and doing so manually could be a problem moving forward.

This is where container orchestration is useful as it helps automate various tasks such as setting up containers, scaling them up and down, connecting them to each other, and ensuring everything is operating well.

Kubernetes and Docker Swarm are two of the most popular container orchestration tools. Both assist you in many ways to manage containers, but they are different and are built for different use cases.

So, which one should you choose? Let's find out.

What is container orchestration and why is it important?

Developing applications using containers is normally straightforward at the start. You typically have only a couple of containers on your local machine, or on one server, using tools like the one provided by Docker to manage them. However, as you build your application, you may have dozens, or hundreds, of containers distributed across multiple servers. When you get to that stage, it is impractical to work

through everything manually, as it is too time-consuming and mistakes are easy to make.

This is where the concept of container orchestration comes into play. This is a system that helps to automate the entire process of deploying containers — starting them, connecting them together, scaling them according to demand, and keeping them running. Instead of deploying each aspect manually, orchestration will ensure your container deployments are done in an intelligent, cost-effective, and uniform way.

DevOps workflows are about achieving everything at speed with consistency and reliability. Orchestration tools help the team to:

- Eliminate manual deployment of containers on all environments for every release.
- Scale up or down based on traffic demand and requirements.
- Load balance between containers so that no container gets overworked with too much traffic.
- Manage the networking environment regardless of the number or type of containers.
- Monitor the health of each container and restart or replace any failed containers.

Initially, a straightforward tool like Docker Compose will suffice if

you're managing a small number of containers. But as soon as you start dealing with many servers, regular coding updates, increased traffic, or a microservices-based architecture, container orchestration will be more than something nice to have — it will be necessary. Without it, it will be hard to have complex applications run reliably, even when you start things small and scale from there.

What is Kubernetes?

Kubernetes, abbreviated as K8s, began as a Google-propelled project. It is derived from the internal Google container orchestration system, Borg, which manages containers at scale. In 2014, Google open sourced Kubernetes. Now Kubernetes is maintained by the Cloud Native Computing Foundation (CNCF) and has become the leading container orchestration system in the world.

So, what makes Kubernetes so powerful?

The architecture of Kubernetes is the foundation of the platform. The smallest unit you can deploy is a pod, which is deployed on nodes (the worker machine in the cluster). The control plane is where all the magic happens — it tells pods where to run and monitors the system's health, while ensuring the latter is always in sync with the desired state.